Documentazione Simulatore RMFS

Contents

1 Assunzioni **2**

2 Descrizione Generale **2**

3 Struttura del Codice **2**

4 Parametri di Input **2**

5 Entità del Dominio **4**
- 5.1 Order 4
- 5.2 Pod 4
- 5.3 Robot 4
- 5.4 Workstation 4
- 5.5 Task e Visit 5
- 5.6 Enumerazioni 5

6 Warehouse Layout **5**

7 Simulator Engine **6**
- 7.1 Event Loop 6
- 7.2 Generazione Ordini 6

8 Event Handlers **6**

1 Assunzioni

Le seguenti assunzioni semplificative delimitano il campo di applicazione del simulatore.

- Le missioni di replenishment non sono considerate, pertanto non si modellano vincoli di inventario. Ogni pod contiene un insieme limitato di SKU, ma non si conosce la quantità esatta né il volume di ciascun articolo.
- Aspetti tecnici dei robot, come gestione dell'energia e routing a basso livello, sono fuori dal campo di questo simulatore e non vengono modellati.

2 Descrizione Generale

Il simulatore ad eventi discreti emula la dinamica operativa di un RMFS (Robotic Mobile Fulfillment System) in stile Amazon, basato su approccio *parts-to-picker*: i robot portano i pod alle workstation, dove gli operatori prelevano gli item degli ordini.

Il simulatore supporta due modalità decisionali:

1. **Ottimizzazione ciclica:** ogni Δt_{opt} viene risolto un modello matematico per la generazione e il sequenziamento dei task.
2. **Policy reattive:** decisioni online basate su euristiche con lookahead limitato, attivate al verificarsi degli eventi.

In entrambe le modalità, l'assegnamento fisico del task a un robot è sempre gestito tramite policy.

3 Struttura del Codice

Il progetto è organizzato su tre livelli logici: `core` (entità e strutture dati condivise), `sim` (motore DES e handler degli eventi), `opt` (policy di assegnazione).

Percorso	**Ruolo**
`run_simulation.py`	Punto di ingresso: configura il logging, costruisce il warehouse, avvia la simulazione
`config.py`	Parametri numerici centralizzati
`scripts/core/enums.py`	Enumerazioni per stati discreti e tipi di eventi
`scripts/core/entities.py`	Dataclass per `Order`, `Pod`, `Robot`, `Workstation`, ecc.
`scripts/core/queues.py`	Coda con priorità generica (min-heap con lookup O(1))
`scripts/core/warehouse.py`	Layout fisico, generazione entità, utilities di distanza
`scripts/sim/Simulator.py`	Motore DES principale: clock, event loop, dispatch table
`scripts/sim/event_handler.py`	Funzione handler per ciascun tipo di evento
`scripts/opt/policies.py`	Policy euristiche di assegnazione

Table 1: Struttura dei folder e dei file del progetto.

4 Parametri di Input

La tabella seguente raccoglie i parametri di configurazione del simulatore. I parametri che definiscono la generazione degli ordini e i tempi di processamento dei pod alle workstation hanno dominio definito da [BJS24][1].

I valori numerici presenti sono quelli adottati da[BJS24] o fissati sulla base della letteratura; i parametri con `???` sono ancora da definire.

[1] Repository github `https://github.com/alexschmid3/warehouse-task-assignment`.

Parametro	**Dominio**	**Descrizione**
Layout e capacità		
`NUM_PODS`	???	Numero totale di pod (`GRID_ROWS` × `GRID_COLS`)
`NUM_SKUS`	???	Numero di SKU distinti
`NUM_ROBOTS`	???	Numero di robot mobili autonomi
`NUM_WORKSTATIONS`	???	Numero di postazioni di picking
`GRID_ROWS / GRID_COLS`	??? / ???	Dimensioni della griglia pod
`WS_ORDER_CAPACITY`	3	Numero massimo di ordini aperti per postazione
`WS_WORKLOAD_CAPACITY`	???	Numero di task rilasciati per workstation
Tempi operativi		
`ROBOT_SPEED`	30.0	Velocità robot (celle/min, $\approx$ 0,5 m/s)
`POD_PROCESS_TIME`	5 s	Tempo di processo di un pod alla workstation
`ITEM_PROCESS_TIME`	5 s	Tempo di picking di un item alla workstation
Generazione ordini		
`INTERARRIVAL_TIME_ORDER`	$\left\{\frac{12}{35}, \frac{6}{25}, \frac{3}{10}, \frac{3}{40}, \frac{1}{5}, \frac{3}{50}, \frac{3}{100}, \frac{3}{250}\right\}$	Tempo medio di interarrivo ordini (min)
`PROB_1_ITEM_ORDER`	0.5	Probabilità che un ordine contenga un solo item
`GEO_DIST_PARAM_ORDER`	$\{0.25, 0.35, 0.45, 0.65\}$	Parametro geometrico per ordini multi-articolo
Controllo simulazione		
`TIME_HORIZON`	???	Durata totale della simulazione (min)
`DELTA_T_OPT`	15 min	Intervallo di chiamata dell'ottimizzatore

Table 3: Parametri numerici del simulatore divisi per funzionalità.

5 Entità del Dominio

Le entità sono implementate come dataclass Python e rappresentano lo stato persistente del sistema durante la simulazione. Le principali sono: `Order`, `Pod`, `Robot`, `Workstation`, `Task`, `Visit`.

5.1 Order

Traccia un ordine cliente dall'arrivo al completamento. Il campo `items_pending` viene decrementato man mano che gli item vengono prelevati.

Campo	Tipo	Descrizione
`order_id`	`int`	Identificatore univoco
`arrival_time`	`float`	Tempo di arrivo dell'ordine
`order_size`	`int`	Numero di item richiesti
`items_required`	`list[int]`	Lista completa degli item richiesti
`items_pending`	`list[int]`	Item non ancora prelevati
`workstation_id`	`int | None`	Workstation assegnata (`None` se non ancora assegnato)
`status`	`OrderStatus`	Stato del ciclo di vita: `BACKLOG → WAITING → OPEN → CLOSED`

5.2 Pod

Rappresenta un pod di stoccaggio. Quando idle, il pod si trova alla propria `storage_location`; quando in missione, il suo stato diventa `BUSY`.

Campo	Tipo	Descrizione
`pod_id`	`int`	Identificatore univoco
`storage_location`	`tuple[int,int]`	Cella di stoccaggio nella griglia
`items`	`list[int]`	SKU attualmente contenuti nel pod
`status`	`PodStatus`	`IDLE` o `BUSY`

5.3 Robot

Ogni robot è sempre in uno dei due stati: `IDLE` (disponibile) o `BUSY` (in esecuzione di un task). Il campo `current_task_id` è `None` quando il robot è libero.

Campo	Tipo	Descrizione
`robot_id`	`int`	Identificatore univoco
`position`	`tuple[int,int]`	Posizione corrente nella griglia
`current_task_id`	`int | None`	Task in esecuzione (`None` se idle)
`status`	`RobotStatus`	`IDLE` o `BUSY`

5.4 Workstation

La workstation gestisce due buffer separati: uno per i pod (`pod_buffer`) e uno per gli ordini in attesa di apertura (`order_buffer`). I task vengono prima rilasciati (`pending_tasks`) e poi allocati a un robot (`active_tasks`).

Campo	Tipo	Descrizione
`workstation_id`	`int`	Identificatore univoco
`order_capacity`	`int`	Massimo ordini aperti simultaneamente
`workload_capacity`	`int`	Massimo task rilasciati per workstation
`position`	`tuple[int,int]`	Coordinate nella griglia
`opened_orders`	`set[int]`	Ordini attualmente in lavorazione
`pod_buffer`	`list[int]`	Pod in attesa (escluso quello attivo)
`order_buffer`	`list[int]`	Ordini in attesa di apertura
`pending_tasks`	`list[Visit]`	Task rilasciati ma non ancora allocati
`active_tasks`	`list[Visit]`	Task correntemente allocati a un robot
`status`	`WorkstationPickingStatus`	`IDLE` o `BUSY`

5.5 Task e Visit

Un `Task` è una missione assegnata a una coppia pod-robot, composta da una sequenza ordinata di fermate (`stops`). Ogni fermata è una `Visit`, che specifica la workstation di destinazione, gli ordini serviti (`orders`) e gli item da prelevare (`items`). Il campo `priority` del task determina l'ordine di dispatching nella coda.

5.6 Enumerazioni

`OrderStatus`, `RobotStatus`, `PodStatus` e `WorkstationPickingStatus` definiscono gli stati discreti delle entità, rendendo esplicite le transizioni di stato nella logica di simulazione.

6 Warehouse Layout

Il warehouse è generato a partire da `grid_rows` e `grid_cols`. I pod sono disposti in una griglia rettangolare; ogni due colonne di pod viene inserita una strada di larghezza due celle per consentire il transito dei robot. Le dimensioni fisiche risultanti sono:

$$X = \text{grid_rows} + 2\left\lfloor \frac{\text{grid_rows} - 1}{2} \right\rfloor + 2 \cdot \text{MARGIN} - 1, \qquad Y = \text{grid_cols} + 2 \cdot \text{MARGIN} - 1$$

Le workstation vengono distribuite sul perimetro, preferibilmente sul lato inferiore in modo simmetrico. I robot partono da posizioni casuali non sovrapposte. Il tempo di percorrenza tra due celle è calcolato come distanza di Manhattan divisa per `robot_speed`; questa è l'unica dipendenza dalla velocità nel codice.

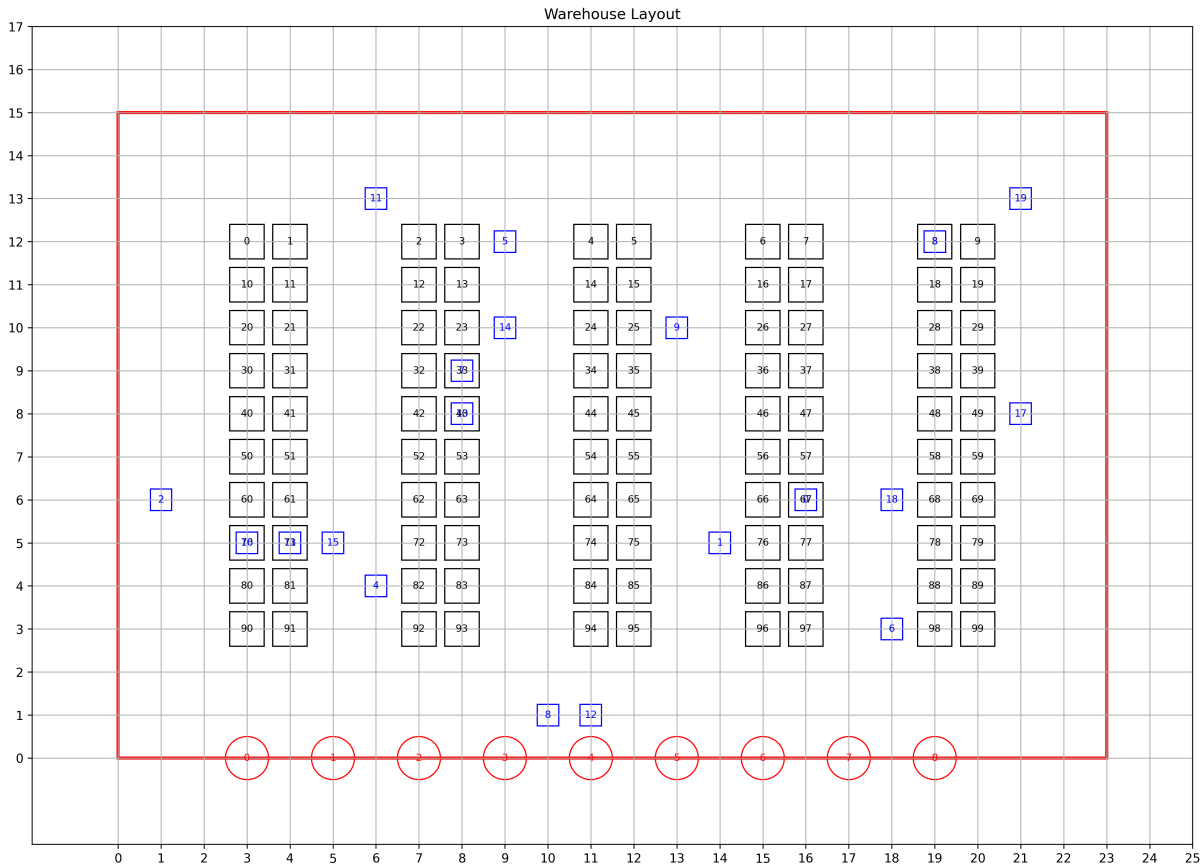

Figure 1: Esempio di layout creato con `GRID_ROWS` = 10, `GRID_COLS` = 10, `NUM_ROBOTS` = 20 e `NUM_WORKSTATION` = 9.
In nero sono le `storage_location` di ciascun pod, in rosso quelle delle workstation e in blu le posizioni iniziali dei robot.

7 Simulator Engine

Il `Simulator` è il contenitore di tutto lo stato della simulazione e guida il loop degli eventi. La tabella seguente descrive i suoi attributi principali.

Attributo	Tipo	Descrizione
`RNG`	`numpy.Generator`	Generatore casuale per riproducibilità
`order_gen_config`	`list[float]`	`[interarrival_time, prob_1_item, geo_param]`
`optimization_enabled`	`bool`	`False` → policy euristiche; `True` → ottimizzatore
`warehouse_status`	`Warehouse`	Ambiente fisico e stato di tutte le entità
`current_time`	`float`	Tempo di simulazione corrente (minuti)
`future_events`	`PriorityQueue[Event]`	Eventi futuri ordinati per timestamp di occorrenza
`orders_in_system`	`PriorityQueue[Order]`	Ordini ricevuti; gli ordini `BACKLOG` hanno priorità
`task_counter`	`int`	Contatore incrementale per gli ID dei task
`released_tasks`	`PriorityQueue[Task]`	Task pronti per l'assegnazione a un robot
`scheduled_tasks`	`dict[int, PriorityQueue[Task]]`	Task programmati per workstation dall'ottimizzatore (usato solo se `optimization_enabled` è `True`)

7.1 Event Loop

Il loop principale segue la struttura classica di un DES ed è definito dal metodo `run_simulation` della classe `Simulator`:

1. Pop dell'evento con timestamp minore da `future_events`.
2. Avanzamento di `current_time` al tempo dell'evento.
3. Dispatch all'handler corrispondente tramite la dispatch table.
4. Ripeti fino a esaurimento della coda o raggiungimento di `TIME_HORIZON`.

7.2 Generazione Ordini

La generazione degli ordini segue il modello di Barnhart et al [BJS24]:

- Con probabilità `PROB_1_ITEM_ORDER`, l'ordine contiene un solo item.
- Altrimenti il numero di item è $\text{Geometric}(p) + 2$, con $p =$ `GEO_DIST_PARAM_ORDER`.
- Gli indici SKU sono estratti da una normale troncata centrata in $N/2$ con $\sigma = N/6$, dove $N =$ `NUM_SKUS`.

8 Event Handlers

Ogni `EventType` è associato a una funzione handler con firma `handler(event, sim)`, che legge e modifica lo stato della simulazione tramite l'oggetto `sim`. La tabella riassume lo stato di implementazione corrente.

Evento	Stato	Responsabilità
`ARRIVAL_ORDER`	**DONE**	Genera ordine, lo inserisce in `orders_in_system`, programma il prossimo arrivo; se senza ottimizzatore, assegna alla workstation meno carica.
`OPEN_ORDER`	**DONE**	Imposta l'ordine a `OPEN`, lo aggiunge a `opened_orders`; se il pod buffer non è pieno, triggera `RELEASE_TASK`.
`RELEASE_TASK`	**DONE**	Se senza ottimizzatore, genera i task tramite policy, li inserisce in `released_tasks` e aggiorna la workstation.
`RUN_OPTIMIZER`	**TODO**	Chiama l'ottimizzatore esterno per assegnare gli ordini in backlog e generare i task.
`START_TASK`	**TODO**	Assegna un robot idle al task; imposta robot e pod a `BUSY`.
`ARRIVAL_POD_WST`	**TODO**	Gestisce l'arrivo del pod alla workstation; lo aggiunge al `pod_buffer` e avvia il picking se la workstation è `IDLE`.
`START_PICKING`	**TODO**	Avvia la sessione di picking; imposta la workstation a `BUSY` e programma `END_PICKING`.
`END_PICKING`	**TODO**	Completa il picking; decrementa `items_pending` per gli ordini coinvolti.
`CLOSE_ORDER`	**TODO**	Imposta l'ordine a `CLOSED` quando tutti gli item sono stati prelevati; libera lo slot nella workstation.
`RETURN_POD`	**TODO**	Riporta il pod alla `storage_location`; imposta robot e pod a `IDLE`.

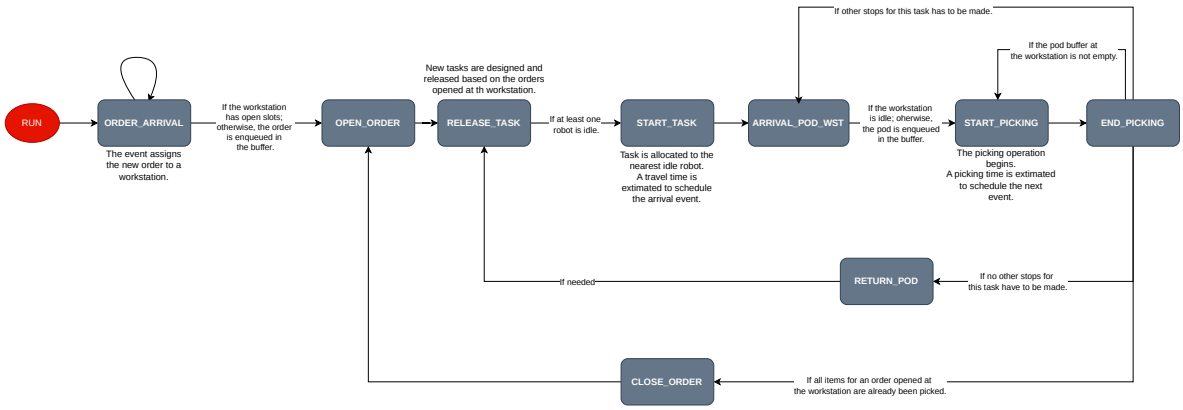

Figure 2: Grafo per mostrare la relazione tra gli eventi quando `optimization_enabled = False`. Le frecce indicano quale evento verrà schedulato.

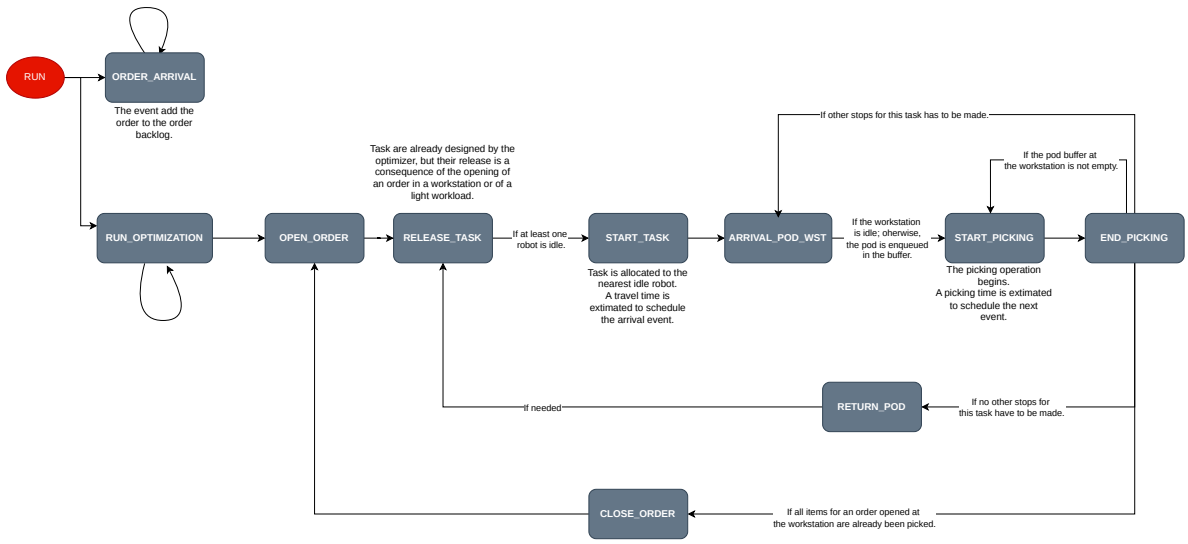

Figure 3: Grafo per mostrare la relazione tra gli eventi quando `optimization_enabled = True`. Le frecce indicano quale evento verrà schedulato.

References

[BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. "Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search". In: *ArXiv* abs/2408.16890 (2024). URL: https://api.semanticscholar.org/CorpusID:272310402.